

SPEC_03_BACKEND.md

Nexus do Mestre — Especificação do Backend

1. Configuração Geral

O backend é uma API REST em Express + TypeScript. Todas as rotas (exceto /auth/login e /auth/register) exigem JWT válido no header Authorization: Bearer <token>. Rotas marcadas com [ADMIN] exigem role admin.

Porta padrão: 3001 **Base URL:** /api/v1

2. Middleware de Auditoria

Criar audit.middleware.ts que intercepta automaticamente requisições POST, PUT, PATCH e DELETE e registra na tabela AuditLog com:

- userId (do token JWT)
- entityType (extraído da rota)
- entityId (extraído dos params)
- action (mapeado do método HTTP)
- oldValue e newValue (serializado como JSON string)
- timestamp

3. WebSocket — Eventos (socket.io)

Eventos emitidos pelo servidor → cliente

Evento	Payload	Descrição
access:new_request	{ userId, username, requestedAt }	Nova solicitação de acesso (apenas admin)
access:approved	{ userId, role }	Notifica usuário que foi aprovado
access:rejected	{ userId, reason }	Notifica usuário que foi recusado
combat:updated	{ combatId, participantId, field, value }	Atualização em tempo real no combate
combat:event	{ event }	Novo evento de combate registrado
combat:round_change	{ round }	Mudança de rodada
notification:general	{ message, type }	Notificações gerais

Eventos emitidos pelo cliente → servidor

Evento	Payload	Descrição
combat:join	{ combatId }	Entrar na sala de combate
combat:leave	{ combatId }	Sair da sala de combate

4. Rotas da API

🔑 Auth (/api/v1/auth)

POST	/register	→ Registrar novo usuário (cria AccessRequest pendente)
POST	/login	→ Login, retorna JWT
GET	/me	→ Dados do usuário autenticado
POST	/logout	→ Invalida sessão (blacklist token opcional)

POST /register body:

json	{ "username": "string (min 3, max 30)", "password": "string (min 6)" }
------	---

POST /login response:

json	{ "token": "jwt_token", "user": { "id", "username", "role", "status", "linkedCharacterId" } }
------	--

👤 Usuários (/api/v1/users) — [ADMIN]

GET	/	→ Listar todos os usuários com status
GET	/pending	→ Listar solicitações pendentes
POST	/approve/:id	→ Aprovar usuário (body: { role, linkedCharacterId? })
POST	/reject/:id	→ Recusar usuário (body: { reason? })
PUT	/:id	→ Atualizar role ou linkedCharacterId
DELETE	/:id	→ Remover usuário
POST	/:id/block	→ Bloquear usuário
POST	/:id/unblock	→ Desbloquear usuário

🎭 Personagens (/api/v1/characters)

GET	/	→ [ADMIN] Listar todos [PLAYER] Listar revelados + próprios
GET	/groups	→ Listar grupos de personagens
POST	/groups	→ [ADMIN] Criar grupo
PUT	/groups/:id	→ [ADMIN] Editar grupo
DELETE	/groups/:id	→ [ADMIN] Deletar grupo
GET	/:id	→ Ver personagem (verificar permissão)
POST	/	→ [ADMIN] Criar personagem
PUT	/:id	→ [ADMIN] Editar personagem completo
PATCH	/:id/vitals	→ [ADMIN PLAYER_OWNER] Atualizar PV/SAN/PE
PATCH	/:id/conditions	→ [ADMIN] Atualizar condições
POST	/:id/skills	→ [ADMIN] Adicionar perícia
PUT	/:id/skills/:skillId	→ [ADMIN] Editar perícia
DELETE	/:id/skills/:skillId	→ [ADMIN] Remover perícia
POST	/:id/abilities	→ [ADMIN] Adicionar habilidade/ritual
PUT	/:id/abilities/:abilityId	→ [ADMIN] Editar habilidade
DELETE	/:id/abilities/:abilityId	→ [ADMIN] Remover habilidade

🐉 Monstros (/api/v1/monsters) — [ADMIN]

GET	/	→ Listar monstros
GET	/:id	→ Ver monstro
POST	/	→ Criar monstro
PUT	/:id	→ Editar monstro
DELETE	/:id	→ Deletar monstro
POST	/:id/attacks	→ Adicionar ataque
PUT	/:id/attacks/:attackId	→ Editar ataque
DELETE	/:id/attacks/:attackId	→ Remover ataque
POST	/:id/abilities	→ Adicionar habilidade
PUT	/:id/abilities/:abilityId	→ Editar habilidade
DELETE	/:id/abilities/:abilityId	→ Remover habilidade
POST	/:id/token	→ Upload de token

🌍 Ambientes (/api/v1/environments)

GET	/	→ [ADMIN] Todos [PLAYER] Revelados
GET	/:id	→ Ver ambiente (verificar permissão)
POST	/	→ [ADMIN] Criar ambiente
PUT	/:id	→ [ADMIN] Editar ambiente
DELETE	/:id	→ [ADMIN] Deletar ambiente
POST	/:id/images	→ [ADMIN] Upload de imagem
DELETE	/:id/images/:imageId	→ [ADMIN] Remover imagem
PATCH	/:id/reveal	→ [ADMIN] Revelar/ocultar

📄 Documentos (/api/v1/documents)

GET	/	→ [ADMIN] Todos [PLAYER] Revelados
GET	/:id	→ Ver documento (verificar permissão)
POST	/	→ [ADMIN] Criar documento
PUT	/:id	→ [ADMIN] Editar documento
DELETE	/:id	→ [ADMIN] Deletar documento
POST	/:id/images	→ [ADMIN] Upload de imagem
DELETE	/:id/images/:imageId	→ [ADMIN] Remover imagem
PATCH	/:id/reveal	→ [ADMIN] Revelar/ocultar

⚔️ Combate (/api/v1/combat)

GET	/	→ [ADMIN] Listar combates
GET	/active	→ Listar combates ativos
GET	/:id	→ Ver combate (participantes + eventos)
POST	/	→ [ADMIN] Criar combate (body: { name, participants[] })
POST	/:id/participants	→ [ADMIN] Adicionar participante ao combate
DELETE	/:id/participants/:participantId	→ [ADMIN] Remover participante
PATCH	/:id/participants/:participantId/vitals	→ Atualizar PV/SAN/PE (emite socket event)
PATCH	/:id/participants/:participantId/conditions	→ Atualizar condições
POST	/:id/events	→ Registrar evento de combate
POST	/:id/next-round	→ [ADMIN] Avançar rodada
POST	/:id/finish	→ [ADMIN] Encerrar combate
DELETE	/:id	→ [ADMIN] Deletar combate

PATCH vitals body:

json	{ "field": "pv san pe", "value": -10, "description": "Dano de mordida do fungo", "sourceId": "participantId_opcional" }
------	--

📋 Notas de Sessão (/api/v1/sessions) — [ADMIN]

GET	/	→ Listar notas
GET	/:id	→ Ver nota
POST	/	→ Criar nota
PUT	/:id	→ Editar nota
DELETE	/:id	→ Deletar nota

📊 Log de Auditoria (/api/v1/audit) — [ADMIN]

GET	/	→ Listar logs (query: entityType, entityId, userId, from, to)
GET	/entity/:type/:id	→ Logs de uma entidade específica

5. Tratamento de Erros

Todos os erros devem retornar no formato:

json	{ "error": true, "message": "Descrição do erro", "code": "ERROR_CODE" }
------	---

Códigos de status:

- 400 → Bad Request (validação falhou)
- 401 → Unauthorized (sem token ou token inválido)
- 403 → Forbidden (sem permissão)
- 404 → Not Found
- 409 → Conflict (ex: username já existe)
- 500 → Internal Server Error

6. Seed Inicial

O arquivo prisma/seed.ts deve criar:

1. Usuário admin com credenciais das variáveis de ambiente ADMIN_USERNAME e ADMIN_PASSWORD